ЛАБОРАТОРНАЯ РАБОТА • 07.10.2026

Внешний учебный тренажер из трех заданий

Интеграция с Moodle через LTI 1.3 и передача итоговой оценки через Assignment and Grade Services (AGS).

Результат работы
Студент открывает тренажер из элемента курса Moodle, отвечает на три задания и видит итоговый балл. Этот же балл появляется у этого студента в нужной колонке журнала Moodle.

Лабораторная адаптирует предложенный проект к стенду без LRS. В упражнении используются только запуск LTI, проверенная идентичность пользователя, серверный расчет и AGS. Историю попыток хранит сам тренажер.

Что создаем	Что проверяем
Три собственных задания с коротким ответом	Нулевой, частичный и максимальный результат
Сервер Node.js на предоставленном каркасе	Пользователь и роль берутся из запуска Moodle
Интерфейс с повторными попытками	Новая попытка отличается от повтора отправки
Передачу оценки в существующий элемент курса	Правильный балл у правильного студента

Материалы к лабораторной

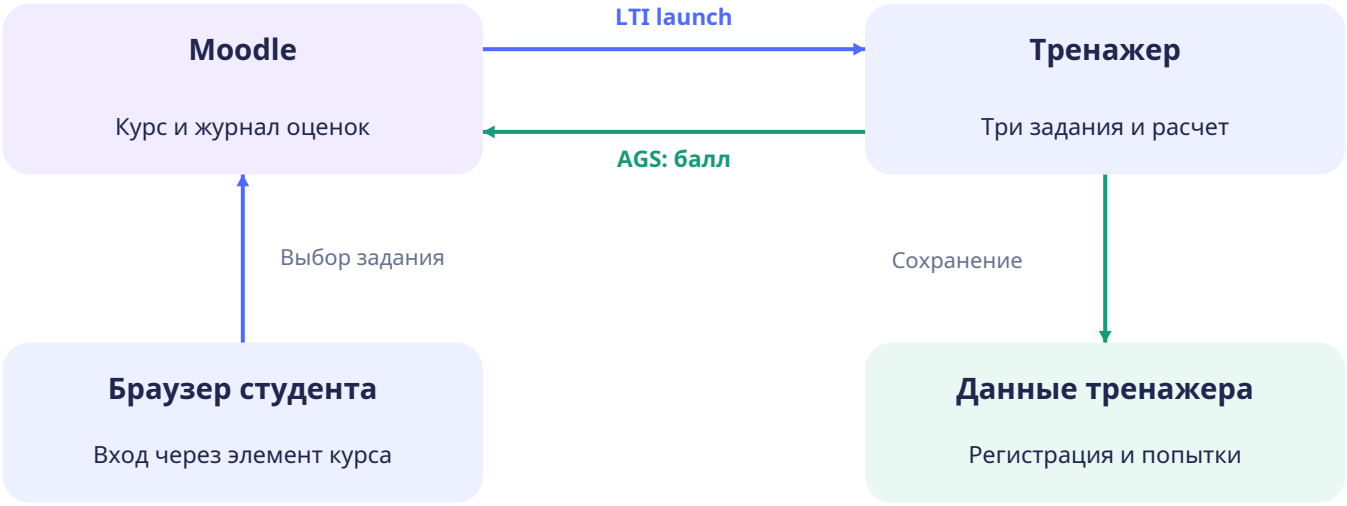
Moodle_LTI_Three_Tasks_Starter.zip - приложение, фиксированные зависимости, конфигурация MongoDB, пример Nginx и тесты. Демонстрационная версия уже содержит три вопроса; ваша задача - заменить их и доказать корректность своей интеграции.

Рекомендуемая организация: два занятия по 90 минут после подготовки стенда. Работа в парах или группах по 3-4 человека; каждый участник проходит короткую индивидуальную защиту.

В инструкции есть отдельный маршрут преподавателя для регистрации инструмента, включая Moodle 3.7. Студентам с готовым стендом достаточно пройти шаги разработки и проверки.

1 • МОДЕЛЬ ВЗАИМОДЕЙСТВИЯ

Как устроено упражнение



1. Запуск. Студент входит в Moodle и нажимает на элемент «Внешний инструмент». Moodle запускает зарегистрированное приложение по LTI. Библиотека проверяет сообщение запуска [1, 5].

2. Работа. Тренажер получает пользователя, роли, контекст курса и идентификатор ресурса. Браузер передает ответы; сервер проверяет их и сохраняет попытку.

3. Оценка. Сервер обращается к сервису оценок Moodle. Для текущего элемента курса используется адрес **line item** из проверенного запуска [2].

Два разных балла
10 баллов - максимум демонстрационного тренажера. Рубрика на 100 баллов в конце этой инструкции оценивает качество самой лабораторной, а не ответы на эти три вопроса.

Правило учебного каркаса	Поведение
Последняя завершенная попытка	Ее балл заменяет прежний балл в колонке Moodle
Повтор отправки	Сохраняет одну и ту же учебную попытку
Неподтвержденная оценка	Сначала повтор отправки, затем новая попытка
Роль преподавателя	Просмотр заданий; отправка себе оценки запрещена

Журнал Moodle хранит итоговую оценку элемента. Ответы и история попыток этого внешнего приложения хранятся в его папке data. Эти данные не становятся историей ответов стандартного теста Moodle автоматически.

2 • ДО НАЧАЛА РАБОТЫ

Выберите маршрут и подготовьте стенд

Маршрут А: преподаватель уже выдал рабочий инструмент

Получите архив, адрес тренажера, элемент курса и способ обновления кода на стенде. Сначала откройте элемент под тестовой учетной записью студента. Перейдите к разделам 8-13: задания, расчет, интерфейс и проверки.

Маршрут Б: инструмент надо зарегистрировать

Сначала выполните разделы 3-7 вместе с администратором Moodle. Для регистрации необходимы соответствующие права; обычной роли студента недостаточно. Преподаватель готовит регистрацию один раз на экземпляр приложения.

Нужно	Проверка готовности
Moodle с поддержкой LTI 1.3	Есть выбор LTI 1.3 в настройках инструмента; он появился в Moodle 3.7 [3]
Курс и две тестовые учетные записи	Оба пользователя записаны на курс с ролью студента
Node.js 24 или новее	node --version показывает v24... или более новую поддерживаемую версию
MongoDB	Сервис из compose.yaml или готовая база преподавателя
Домен и HTTPS тренажера	Браузер и сервер Moodle могут открыть этот адрес
Один процесс Node.js	Файловый журнал рассчитан на один экземпляр процесса

Для нескольких групп
Выдайте отдельный экземпляр приложения и регистрацию инструмента каждой группе. Несколько студентов одной группы работают с одним экземпляром, но получают отдельные попытки. Разные наборы вопросов не загружайте в один общий процесс одновременно.

MongoDB здесь нужна библиотеке LTI для регистрации, ключей и сведений о запусках. Она не является LRS. В каркасе используются ltijs 7.0.7 и Express 5.2.1; команды npm ci опираются на приложенный package-lock.json.

3 • ФАЙЛЫ И КОМАНДЫ

Распакуйте и проверьте каркас

1. Распаковка. Распакуйте архив в отдельную папку. Внутри находится каталог moodle-lti-three-tasks-lab. Откройте терминал в нем.

```
cd moodle-lti-three-tasks-lab
node --version
npm ci
cp .env.example .env
docker compose up -d mongo
npm test
```

Команда docker compose нужна, если MongoDB еще не предоставлена преподавателем. Docker должен быть установлен и запущен. Строка подключения к готовой базе задается в MONGO_URL.

Файл	Назначение / что менять
src/grading.mjs	Три вопроса, ответы, веса и функция grade: основная работа студента
public/index.html, app.js, style.css	Форма, обращение к API и оформление
src/tool.mjs	LTI-сессия, роли, сохранение и отправка AGS; изучить поток
src/server.mjs	Чтение конфигурации и запуск приложения
.env	Адреса и идентификаторы реального стенда; не сдавать публично
test/*.test.mjs	Расчет и имитация подписанного запуска с AGS
compose.yaml; nginx.example.conf	MongoDB и пример HTTPS reverse proxy
data/	Появится при отправке ответов; личные попытки

Контрольная точка
Базовые тесты должны завершиться с pass 4, fail 0. В интеграционном тесте специально имитируется сбой AGS, поэтому одна строка «AGS не подтвердил оценку: HttpError» ожидаема.

npm test не требует запущенного Moodle или MongoDB: тестовый сценарий использует память и имитацию платформы. Обычный npm start использует настоящую MongoDB.

4 • АДРЕСА ПРИЛОЖЕНИЯ

Настройте HTTPS и первый запуск

1. Заполните .env. Укажите HTTPS-адрес тренажера без завершающего слеша, название и URL базы. Пока оставьте MOODLE_CLIENT_ID и MOODLE_DEPLOYMENT_ID пустыми: сначала Moodle должен создать регистрацию.

```
PORT=3000
PUBLIC_URL=https://trainer.example.org
APP_LABEL=Тренажер группы 01
MONGO_URL=mongodb://127.0.0.1:27017/lti_lab
DATA_DIR=./data
MOODLE_CLIENT_ID=
MOODLE_DEPLOYMENT_ID=
```

example.org - пример, замените его своим доменом. Адреса MOODLE_ISSUER, MOODLE_AUTH_URL, MOODLE_TOKEN_URL и MOODLE_JWKS_URL также замените; окончательно сверьте их с реквизитами Moodle на следующем этапе.

2. Проксирование. Передайте приложению HTTPS-трафик через имеющийся reverse proxy. В nginx.example.conf приведен пример; подставьте свой домен и пути уже выданного сертификата. Node.js слушает HTTP на порту 3000 за прокси.

```
location / {
    proxy_pass http://127.0.0.1:3000;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header X-Forwarded-For
        $proxy_add_x_forwarded_for;
}
```

3. Запустите. В терминале выполните npm start. Оставьте процесс работающим; в другом терминале проверьте адрес.

```
curl https://trainer.example.org/healthz
# Ожидается: {"ok":true,"configured":false}
```

Контрольная точка
Адрес /healthz доступен из браузера и с сервера Moodle. При использовании HTTPS в Moodle тренажеру также нужен HTTPS [4]. Пока Client ID пуст, попытка запуска курса не должна быть рабочей.

5 • НАСТРОЙКА MOODLE

Создайте регистрацию LTI 1.3

Выполняет преподаватель или администратор. В администрировании сайта откройте: «Плагины > Элементы курса > Внешний инструмент > Управление инструментами». Выберите ручное добавление / «Настроить инструмент вручную». В новых версиях возможна также настройка на уровне курса [4].

Поле Moodle	Значение для этого каркаса
Название	Тренажер группы 01
Версия LTI	LTI 1.3
URL инструмента / Tool URL	https://trainer.example.org/lti/launch
Initiate login URL / Инициация входа	https://trainer.example.org/lti/login
Redirection URI(s)	https://trainer.example.org/lti/launch
Тип открытого ключа, если поле есть	Keyset URL / набор открытых ключей
Public keyset URL, если поле есть	https://trainer.example.org/lti/keys
Контейнер запуска	Новое окно / New window
Выбор содержимого / Deep Linking	Не включать: в каркасе один набор заданий

Отдельная ветка для Moodle 3.7

В форме Moodle 3.7 есть текстовое поле **Public key** для PEM-ключа [6]. Не вставляйте в него адрес /lti/keys или JSON. Для первого сохранения оставьте его пустым; инструмент пока не запускайте. После получения Client ID приложение создаст ключ, и в разделе 6 вы вставите полный PEM.

Разрешите прием оценки

В разделе Services выберите **IMS LTI Assignment and Grade Services**: вариант **Use this service for grade sync only** достаточен для существующей колонки журнала [7]. В настройках конфиденциальности разрешите прием оценок. Имя можно передавать для приветствия; email упражнению не требуется.

Сохранение и реквизиты
Сохраните инструмент и откройте его реквизиты / View configuration details. Скопируйте Platform ID, Client ID, Deployment ID и адреса аутентификации, токена и набора ключей. В разных переводах названия могут отличаться.

6 • ОБРАТНАЯ РЕГИСТРАЦИЯ

Перенесите реквизиты Moodle в .env

Теперь тренажер должен знать, какая платформа вправе его запускать. Используйте значения из созданной регистрации, включая путь подкаталога Moodle. Не заменяйте Platform ID произвольным адресом главной страницы.

Реквизит Moodle	Переменная .env
Platform ID / issuer	MOODLE_ISSUER
Client ID	MOODLE_CLIENT_ID
Deployment ID	MOODLE_DEPLOYMENT_ID
Authentication request URL	MOODLE_AUTH_URL
Access token URL	MOODLE_TOKEN_URL
Public keyset URL платформы	MOODLE_JWKS_URL

```
# Только пример для Moodle в подкаталоге /m:
MOODLE_ISSUER=https://moodle.example.org/m
MOODLE_CLIENT_ID=ЗНАЧЕНИЕ_ИЗ_MOODLE
MOODLE_DEPLOYMENT_ID=ЗНАЧЕНИЕ_ИЗ_MOODLE
MOODLE_AUTH_URL=https://moodle.example.org/m/mod/lti/auth.php
MOODLE_TOKEN_URL=https://moodle.example.org/m/mod/lti/token.php
MOODLE_JWKS_URL=https://moodle.example.org/m/mod/lti/certs.php
```

1. Перезапуск. Остановите Node.js сочетанием Ctrl+C. Снова выполните npm start. Изменения .env вступают в силу при новом запуске процесса.

2. Проверка. /healthz теперь возвращает configured: true. /lti/keys возвращает JSON с публичными ключами. Это проверка настройки приложения, а не доказательство удачного запуска из Moodle.

3. PEM для Moodle 3.7. Получите публичный ключ и вставьте весь текст, включая BEGIN/END PUBLIC KEY, в поле Public key настроек инструмента. Сохраните изменения.

```
curl https://trainer.example.org/tool-public-key   -o tool-public-key.pem
```

Контрольная точка
В новых версиях настроен JWKS URL; в Moodle 3.7 вставлен PEM. Один и тот же Client ID используется с обеих сторон. Если Moodle показывает инструмент как ожидающий активации, активируйте его. Не удаляйте том MongoDB: в нем сохраняются ключи регистрации.

7 • ЭЛЕМЕНТ КУРСА

Добавьте тренажер и выполните первый запуск

1. Добавьте элемент. Включите редактирование курса. Добавьте «Внешний инструмент» либо выберите уже зарегистрированный инструмент в списке элементов. Укажите название «Три задания - группа 01».

2. Выберите тип. Выберите именно созданный преднастроенный инструмент LTI 1.3. Не создавайте обычную URL-ссылку. URL запуска должен совпадать с /lti/launch; ключ потребителя и общий секрет из LTI 1.1 здесь не заполняются.

3. Оценивание. В разделе оценки выберите числовой балл и максимум **10**. Убедитесь, что прием оценок от инструмента разрешен. Если раздела оценки нет, вернитесь к разрешениям инструмента [8].

4. Отображение. Выберите запуск в новом окне. Сохраните элемент. Приложенный интерфейс рассчитан на отдельное окно; его CSP запрещает встраивание в iframe.

5. Учетная запись. Войдите в Moodle отдельной тестовой учетной записью студента, записанного на курс. Для проверки идентичности надежнее отдельный профиль браузера или приватное окно, а не только переключение роли администратора.

6. Запуск. Откройте элемент курса. Убедитесь, что появились приветствие, три вопроса, идентификатор пользователя, контекст курса и идентификатор ресурса.

Проверка	Ожидаемое поведение
Открыть /app напрямую без LTI-сессии	401 и указание открыть тренажер из Moodle
Открыть элемент как студент	Форма активна, роль Learner
Открыть как преподаватель	Форма доступна для просмотра; отправка отключена
Истекли 45 минут или сервер перезапущен	Запустить элемент Moodle заново

Контрольная точка
Первый успешный запуск доказывает связь Moodle > тренажер. Для завершения интеграции надо отдельно выполнить отправку балла и проверить журнал Moodle в разделе 12.

8 • РАБОТА СТУДЕНТОВ

Создайте три собственных задания

Задание. На основе каркаса разработайте внешнее упражнение по выбранной теме. Три задания должны иметь короткие проверяемые ответы. Оставьте идентификаторы q1, q2, q3 и веса 3, 3, 4: сумма равна 10.

1. Выберите тему. Например: HTTP, JavaScript, разметка HTML или лексика иностранного языка. Сформулируйте вопросы так, чтобы правильный ответ не зависел от мнения проверяющего.

2. Измените массив tasks. Откройте src/grading.mjs и замените prompt и answer. В эталоне показана демонстрационная версия; правильные ответы остаются на сервере.

```
export const tasks = [
  {
    id: 'q1',
    prompt: 'HTTP-метод отправки формы с телом?',
    answer: 'post', points: 3
  },
  {
    id: 'q2',
    prompt: 'Какую строку возвращает typeof []?',
    answer: 'object', points: 3
  },
  {
    id: 'q3',
    prompt: 'Сервис LTI для передачи оценок?',
    answer: 'ags', points: 4
  }
];
```

3. Задайте нормализацию. Базовый каркас убирает пробелы по краям и игнорирует регистр. В объяснении задания явно укажите это правило. Правильный ответ должен быть непустой строкой, отличной от __wrong__.

4. Примените изменения. Перезапустите Node.js, запустите npm test и заново откройте элемент Moodle. Форму приложение строит из публичной части массива tasks автоматически.

Контрольная точка
В Network > /api/session браузер получает id, prompt и points. Поле answer из массива tasks туда не передается. При этом история собственных ответов пользователя доступна в данных его сессии.

9 • СЕРВЕРНАЯ ПРОВЕРКА

Разберите расчет итогового балла

Функция grade принимает объект ответов, проверяет его структуру, вычисляет баллы и возвращает итог. В src/grading.mjs уже есть реализация; объясните ее и добавьте собственные проверки для выбранной темы.

```
export function normalize(value) {
  return value.trim().toLowerCase();
}

// После проверки структуры answers:
const items = tasks.map(task => {
  const correct = normalize(answers[task.id]) ===
    normalize(task.answer);
  return {
    id: task.id,
    correct,
    points: correct ? task.points : 0
  };
});

const scoreGiven = items.reduce(
  (sum, item) => sum + item.points, 0
);
const scoreMaximum = tasks.reduce(
  (sum, task) => sum + task.points, 0
);
```

Ситуация	Результат проверки
Все ответы неверны или пусты	0/10
Верны q1 и q3	7/10
Верны все три	10/10
Нет q3; лишний ключ; вместо строки массив	HTTP 400; оценка не отправляется
Ответ длиннее 200 символов	HTTP 400; оценка не отправляется

1. Сверьте тесты. В test/grading.test.mjs есть нулевой, максимальный и частичный результаты, нормализация и некорректный ввод. Добавьте хотя бы один тест, характерный для своих формулировок.

2. Сохраните контракт. Не принимайте готовый scoreGiven от браузера. Полный расчет должен выполняться в серверной функции. Максимум в этом варианте остается равным 10.

Если вам нужен набор допустимых ответов, расширьте серверную проверку осознанно: например, храните массив эквивалентных форм. Тогда измените тестовые данные и пояснение правила, а не только форму ввода.

10 • ФОРМА И API

Проверьте отправляемые данные

1. Изучите интерфейс. В public/app.js форма получает задания от /api/session. После нажатия кнопки браузер собирает ответы, создает requestId и отправляет запрос в собственное приложение.

```
POST /api/submit
Content-Type: application/json
X-CSRF-Token: <токен текущей сессии>

{
  "requestId": "d5d775b2-a480-49b9-bdb4-4dde8dafd14d",
  "answers": {
    "q1": "post",
    "q2": "object",
    "q3": "ags"
  }
}
```

2. Наблюдайте запрос. Откройте DevTools > Network и нажмите «Проверить и отправить в Moodle». Найдите /api/submit. Сверьте JSON-тело с примером: только answers и requestId.

3. Проверьте результат. В ответе приложения найдите current.scoreGiven, current.scoreMaximum, current.status и current.sentAt. sentAt заполняется после успешного ответа сервиса Moodle.

4. Улучшите форму. Добавьте ясные формулировки, видимое число баллов и аккуратное сообщение о результате. Сохраните работу с клавиатурой, подписи полей и область сообщения role="status". Не добавляйте поле ручного ввода идентификатора студента.

Идентичность пользователя
LTI userId может быть непривычным идентификатором и не совпадать с логином Moodle. Сервер берет его из проверенного token.user.id. Имя используется только для приветствия; пользователь и адрес строки журнала не принимаются из JSON формы.

Кнопка «Новая попытка» появляется после подтвержденной отправки. Для нее предусмотрен POST /api/new-attempt с пустым объектом. Она переносит завершенную попытку в историю и разрешает новый ответ.

11 • ПЕРЕДАЧА ОЦЕНКИ

Как каркас вызывает AGS

Откройте обработчик /api/submit в src/tool.mjs. Проследите путь от проверенной сессии к рассчитанному результату и вызову grading.submitScore. Здесь context - объект проверенного запуска библиотеки ltijs 7 [5].

```
const token = context.idToken;
const ags = token.services.assignmentAndGrades;

await context.grading.submitScore(ags.lineItemId, {
  userId: token.user.id,
  scoreGiven: attempt.scoreGiven,
  scoreMaximum: attempt.scoreMaximum,
  activityProgress: 'Completed',
  gradingProgress: 'FullyGraded',
  timestamp: deliveryTimestamp
});
```

Значение	Источник / смысл
userId	Пользователь из проверенного запуска
ags.lineItemId	Адрес существующей строки журнала для элемента курса
scoreGiven / scoreMaximum	Баллы, вычисленные сервером
Completed / FullyGraded	Работа завершена, результат оценен
deliveryTimestamp	Серверное время этой отправки; строго возрастает

Библиотека запрашивает OAuth access token и отправляет сообщение Score. Эти сведения не надо копировать в код браузера. В каркасе используются права записи score в готовую колонку, новая колонка программно не создается.

Масштаб оценки
Для упражнения установите максимум 10 и в приложении, и в элементе Moodle. AGS может передавать отношение scoreGiven/scoreMaximum, которое платформа пересчитывает к максимуму своей колонки [2]. Для контрольной проверки не используйте шкалу 2-5.

Не собирайте адрес AGS вручную из URL Moodle. В частности, адрес line item может содержать query-параметр: добавление /scores простой конкатенацией после всего URL ломает такой адрес. Эту операцию выполняет библиотека.

12 • ПРОВЕРКА В MOODLE

Выполните контрольные сценарии

Используйте тестовые учетные записи, а не выставленные оценки реальных учащихся. После каждого сценария возвращайтесь во вкладку курса и обновляйте журнал оценок. Проверьте строку именно текущего студента и колонку именно этого тренажера.

Сценарий	Тренажер	Журнал Moodle
Студент А: все ответы неверны	0/10; отправка подтверждена	0 в его строке
Новая попытка А: верны q1 и q3	7/10; новая попытка в истории	7 вместо прежнего 0
Новая попытка А: верно все	10/10	10 в той же колонке
Обновить окно или повторить тот же запрос	Нет новой учебной попытки	Нет новой колонки
Студент Б: неверные ответы	0/10; история А не видна	У Б 0, у А остается 10
Преподаватель открывает элемент	Просмотр; отправка отключена	Ему не отправляется балл

1. Уточните путь. Преподаватель открывает курс > «Оценки» > отчет по пользователям / журнал. Студент смотрит свой отчет по оценкам. Названия пунктов зависят от темы и версии.

2. Сопоставьте максимум. Проверьте тип «балл» и максимум 10. Если Moodle показывает 70 при результате 7/10, у колонки, вероятно, максимум 100. Это масштабирование, а не обязательно ошибка расчета.

3. Подтвердите связь. Запишите: учетная запись, название элемента, ожидаемый балл, фактический балл. Не делайте вывод об успехе по одному сообщению интерфейса - нужен результат в журнале.

Контрольная точка
Обязательный результат: у А и Б разные итоги в своих строках одной существующей колонки. История попыток доступна каждому только в его сессии тренажера. Итог соответствует принятому правилу последней завершенной попытки.

13 • ПРОВЕРКИ И СБОИ

Проверьте границы доверия

Запрос без запуска Moodle

```
curl -i https://trainer.example.org/api/session
# Ожидается HTTP 401.
```

Подмена пользователя из браузера

В открытом окне тренажера выполните следующий код в DevTools. Сервер должен отклонить поле userId, даже если сессия действительна. Для этой проверки используйте тестового студента.

```
const s = await fetch('/api/session').then(r => r.json());
const r = await fetch('/api/submit', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'X-CSRF-Token': s.csrf
  },
  body: JSON.stringify({
    requestId: crypto.randomUUID(),
    answers: { q1: 'post', q2: 'object', q3: 'ags' },
    userId: 'чужой-пользователь'
  })
});
console.log(r.status); // 400
```

Дополнительная проверка	Ожидается
POST без CSRF-токена	403; оценки не меняются
POST из роли Instructor	403; оценки не меняются
Лишнее поле scoreGiven в JSON	400; сервер не принимает готовый балл
Повтор подписанного запуска с тем же nonce	Отказ библиотеки; проверяет npm test
Сбой приема AGS	Попытка сохраняется; интерфейс предлагает повтор

Не выключайте общий Moodle ради проверки сбоя. npm test уже включает временную недоступность имитированного сервиса AGS. Режим Offline в браузере проверяет связь браузер > тренажер, но не канал тренажер > Moodle.

14 • ПОВТОРНАЯ ОТПРАВКА

Различайте попытку и доставку оценки

1. Сохранение. Перед вызовом AGS каркас сохраняет ответы и результат в data. Если прием оценки не подтвержден, попытка остается со статусом pending и sentAt: null.

2. Повтор. Кнопка «Повторить отправку» передает тот же requestId и те же ответы. Сервер использует сохраненный расчет. Новая учебная попытка не создается; pending блокирует создание следующей попытки.

3. Подтверждение. После успешного ответа AGS сохраняются status: sent и sentAt. Теперь можно начать новую попытку. Уже подтвержденный повтор того же запроса возвращает прежний результат без нового вызова AGS.

После неоднозначного сетевого сбоя
Moodle мог принять оценку, даже если приложение не получило ответ. Повтор может записать тот же балл еще раз. Поэтому каркас не обещает exactly-once: он сохраняет одну учебную попытку и при новой отправке использует возрастающий timestamp. Следующая попытка не начнется раньше подтверждения предыдущей.

Что сохраняется

Место	Данные
MongoDB	Регистрация платформы, ключи и данные запусков библиотеки
data/<хеш>.json	История и текущая попытка конкретного пользователя и ресурса
Память процесса	Сессия интерфейса на 45 минут и одноразовый билет перехода
Журнал Moodle	Последний принятый итоговый балл элемента курса

После перезапуска Node.js снова откройте элемент Moodle: сессии интерфейса из памяти не переживают перезапуск. Если папка data сохранена, текущая попытка и история будут восстановлены для того же LTI-пользователя и ресурса.

Ограничение учебного каркаса

Запускайте один процесс Node.js для этой папки data. Последовательная обработка защищает от гонок внутри этого процесса. Для нескольких процессов или серверов перенесите журнал и блокировки в общую базу с транзакциями; это отдельное расширение проекта.

15 • ДИАГНОСТИКА

Если что-то не заработало

Симптом	Что проверить
Нельзя открыть /healthz	Процесс Node.js, порт 3000, DNS, сертификат и reverse proxy
Ошибка подключения к MongoDB	docker compose ps; MONGO_URL; доступность базы из процесса
Unregistered platform	Issuer и Client ID; значения из реквизитов; перезапуск после .env
Неверный Deployment ID	Точное значение регистрации Moodle; не логин и не ID курса
Ошибка подписи / набор ключей	В Moodle 3.7 полный PEM; в новых версиях верный JWKS URL; ключи Moodle в .env
Ошибка state / повторный nonce	Заново открыть элемент Moodle; не переотправлять старую форму запуска
TOKEN_TOO_OLD	Синхронизация времени серверов и задержка запуска; открыть элемент заново
Нет AGS или line item	Сервис grade sync, прием оценок, числовая оценка у элемента; затем новый запуск
Оценка не отправлена, HTTP 502	Доступность token/score URL с сервера; разрешение score; журнал процесса
Ошибка 401 на /api/session	Истекла сессия, перезапуск процесса или прямой вход без Moodle
Оценка в другом масштабе	Максимум колонки Moodle: в лабе 10; шкала вместо числового балла
У преподавателя нет отправки	Ожидаемая политика каркаса; открыть настоящую учетную запись студента
После смены ключей не работает вход	Не удалялся ли том MongoDB; обновить PEM или настройки JWKS
Код изменен, но вопросы старые	Перезапуск Node.js и новая загрузка окна; убедиться, что изменен нужный экземпляр

Для диагностики приложения смотрите терминал с npm start или журнал вашего менеджера процессов. Не прикладывайте полный id_token, cookie, ltik или приватные ключи к отчету. Достаточно сообщения ошибки, маршрута, HTTP-статуса и времени проверки.

16 • СДАЧА ЛАБОРАТОРНОЙ

Что представить преподавателю

1. Исходный код. Архив или репозиторий приложения без node_modules, .env, data и ключей. Приложите .env.example с пустыми реквизитами и краткую инструкцию запуска.

2. Описание вопросов. Тема, три формулировки, допустимые ответы, нормализация и веса. Сумма весов - 10. Объясните, что происходит при пустом и некорректном ответе.

3. Подтверждение интеграции. Снимки тренажера и журнала Moodle для тестовых А и Б; таблица ожидаемых и полученных баллов. Имена других пользователей и идентификаторы сессии скройте.

4. Тесты. Результат npm test и собственный дополнительный тест. Отдельно укажите, какие сценарии выполнялись на настоящем Moodle, а какие - на имитации.

5. Краткий разбор. Схема потока «запуск > ответы > расчет > AGS > журнал» и объяснение повторной отправки. Подготовьте индивидуальную демонстрацию изменения одного правила.

Критерий качества лабораторной	Баллы
Три собственных задания и понятный интерфейс	20
Серверная проверка и корректный максимум	20
Запуск LTI, роли и правильная идентичность	20
Баллы А и Б в нужной колонке Moodle	20
Отрицательные проверки и повтор после сбоя	10
Воспроизводимость и индивидуальное объяснение	10
Итого	100

Вопросы для индивидуальной защиты

Откуда взят userId? Почему браузер не должен присылать итоговый балл? Что такое line item? Почему преподаватель не отправляет себе оценку? Чем новая попытка отличается от повтора отправки? Какие данные исчезнут и какие сохранятся после перезапуска Node.js?

17 • ПАМЯТКА ПРЕПОДАВАТЕЛЮ

Как организовать проверку группы

Для первого занятия выдайте уже зарегистрированный стенд. Тогда студенты потратят время на задания, расчет и наблюдение обмена, а не на сертификаты и административные права. Ручную регистрацию можно сделать общей демонстрацией или отдельной задачей сильным группам.

Этап	Время	Результат
Занятие 1: запуск и разбор каркаса	20 мин	Учетная запись и контекст видны
Собственные вопросы и расчет	40 мин	Три вопроса, 0/7/10 и тесты
Интерфейс и первая передача AGS	30 мин	Балл появился в Moodle
Занятие 2: А/Б и повторные попытки	30 мин	Изоляция и правило последней попытки
Отрицательные проверки	25 мин	Нет подмены userId или балла
Взаимная проверка и защита	35 мин	Отчет и индивидуальное объяснение

Единый контракт для массовой проверки

Зафиксируйте q1/q2/q3, веса 3/3/4, максимум 10, формат answers/requestId и маршруты API. Предоставленные тесты проверяют общий контракт, но не качество формулировок вопросов. Качество заданий и понимание кода оценивайте на защите.

Сохранность действующей регистрации
На время курса сохраняйте том MongoDB, папку data и выбранные домены. Для остановки базы достаточно docker compose stop mongo. Не применяйте docker compose down -v к работающему учебному стенду: удаление тома удалит сохраненные ключи.

Не публикуйте клиентский секрет из другой интеграции или пароль Moodle: в этой лабе они не нужны. Пользователь входит в Moodle; LTI-проверки выполняет библиотека. В отчетах достаточно обезличенных контрольных учетных записей.

Каркас проверен на уровне серверного расчета и подписанного обмена с имитацией LTI-платформы. Реальный Moodle, его тема, права, HTTPS и серверные настройки требуют выполнения контрольных шагов на вашем стенде.

18 • ИСТОЧНИКИ И ГРАНИЦЫ

На чем основана инструкция

[1] 1EdTech. LTI Core 1.3

https://www.imsglobal.org/spec/lti/v1p3/

[2] 1EdTech. Assignment and Grade Services 2.0

https://www.imsglobal.org/spec/lti-ags/v2p0

[3] Moodle 3.7: release notes, LTI 1.3 (MDL-62599)

https://docs.moodle.org/dev/Moodle_3.7_release_notes

[4] MoodleDocs. Управление внешними инструментами

https://docs.moodle.org/503/en/Managing_LTI_External_tools

[5] ltijs: официальный репозиторий и документация

https://github.com/Cvmcosta/ltijs

[6] Moodle 3.7: форма ручной регистрации

https://github.com/moodle/moodle/blob/MOODLE_37_STABLE/mod/lti/edit_form.php

[7] Moodle 3.7: исходный код сервиса оценок

https://github.com/moodle/moodle/tree/MOODLE_37_STABLE/mod/lti/service/gradebookservices

[8] MoodleDocs. Элемент курса LTI External tool

https://docs.moodle.org/503/en/External_tool_settings

Версии и метод проверки
Материал подготовлен 07.10.2026. API каркаса зафиксирован на ltijs 7.0.7; примеры старого API ltijs 5 нельзя смешивать с ним. Поведение полей Moodle 3.7 проверено по официальному исходному коду; новые экраны - по MoodleDocs.

Три демонстрационных задания, политика последней попытки, файловый журнал, ограничения по ролям и рубрика являются решениями учебного каркаса. Это не обязательные требования стандартов LTI. В базовом упражнении xAPI и LRS не используются.

Полная инструкция читается вместе с архивом. В PDF показаны ключевые фрагменты и действия; законченные файлы приложения, тестов и конфигурации находятся в архиве.